Sivas Cumhuriyet Üniversitesi
Mobil Programlama I

İleri Düzey Dart Mimarisi ve Dil Özellikleri

Eşzamanlılık, Isolate Modeli ve Sağlam Null Güvenliği Üzerine Kapsamlı Akademı Rehberi

Akademisyen: Doç. Dr. Mesut Polatgil
Takım İsmi: Toys Code
Takım Üyeleri:

- Ali Akbaş - 2024481058
- Emrullah Altay - 2024481005
- Emirhan Yazıcı - 2024481265

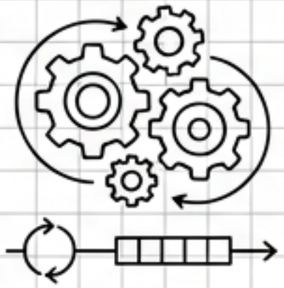
Gemini Notebook

Dijital Laboratuvar İçerik Haritası

Her şey bir Isolate içinde yaşar, hiçbir şey tesadüf değildir. Adım adım mimariyi inşa edelim.

Pillar 1

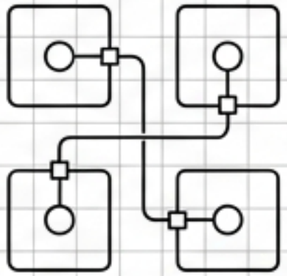
Tek İpliğin Gücü
(Concurrency)



Event Loop, Microtask Queue ve Asenkron API'lerin anatomisi.

Pillar 2

Paralel Evrenlere Kaçış (Isolates)



Paylaşımsız bellek mimarisi, Port iletişimleri ve Worker isolate'lar.

Pillar 3

Kırılmaz Kalkan
(Null Safety)

Milyar dolarlık hatanın çözümü, Tip Evreni ve Derleyici Akış Analizi.

Pillar 4

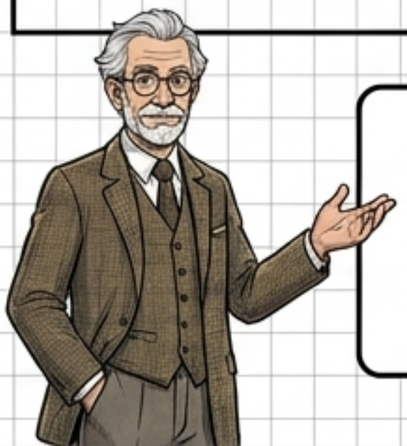
Mimari Araçlar
(Keywords & Versioning)

'late', 'required', Null-Aware operatörler ve geriye dönük uyumluluk.

Gemini Notebook

Concurrency (Eşzamanlılık) Temelleri: Thread vs Isolate

Geleneksel Thread Modeli	Dart Isolate Modeli
• **Bellek:** Paylaşımlı (Shared Memory). • **İletişim:** Doğrudan bellek erişimi. • **Riskler:** Data Race (Veri Yarışı), Deadlocks. • **Mekanizma:** Mutex, Lock ve Semaphore karmaşası.	• **Bellek:** Tamamen Yalıtılmış (Isolated Memory). • **İletişim:** Sadece Portlar üzerinden Mesajlaşma (Message Passing). • **Avantaj:** Kilitlenme yok, Lock mekanizmalarına ihtiyaç yok. • **Mekanizma:** Her Isolate kendi Olay Döngüsüne (Event Loop) sahiptir.



Uzman Tavsiyesi:

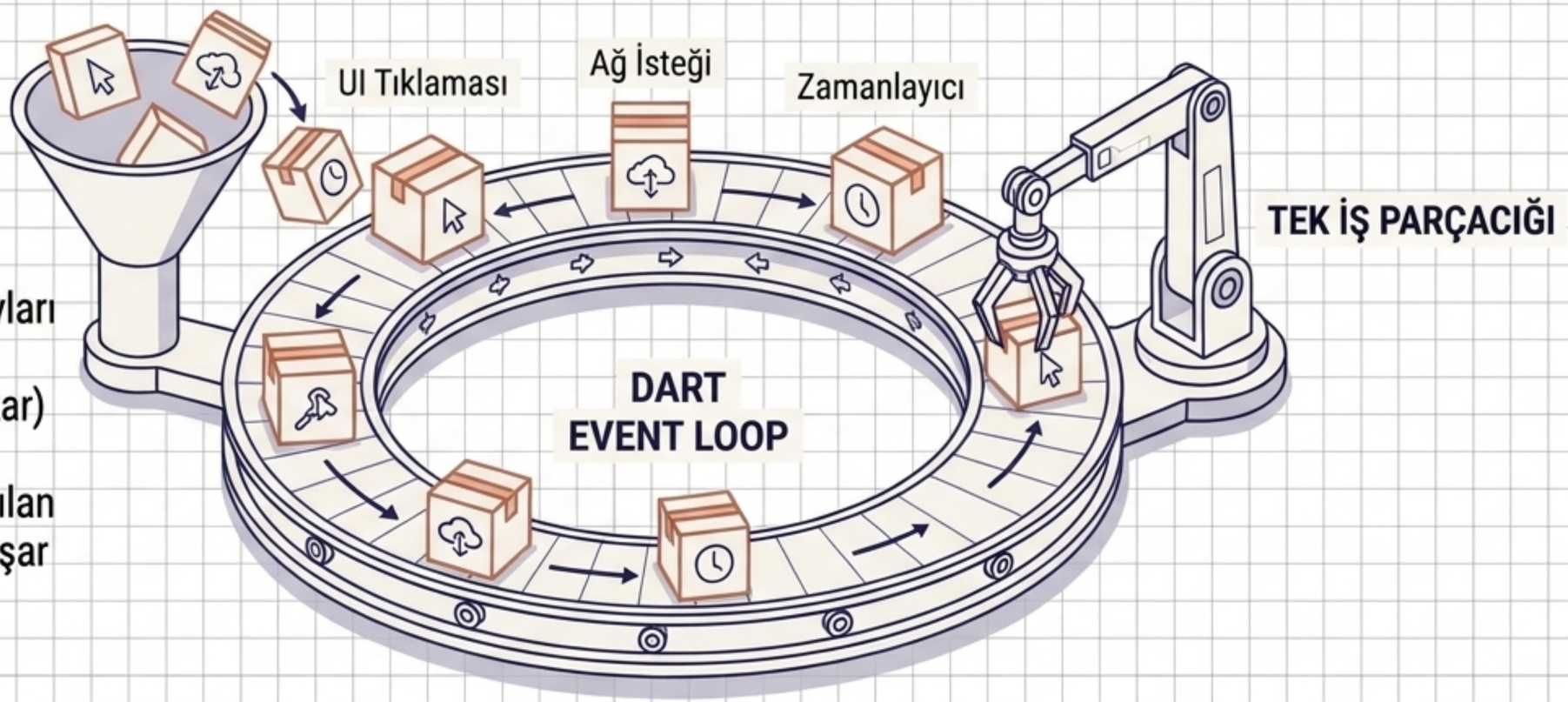
Dart, paylaşımlı durumu ortadan kaldırarak eşzamanlılığı hataya kapalı hale getirir. Bellek paylaşılmaz, veriler kopyalanarak veya aktarılarak iletilir.

Gemini Notebook

Olay Döngüsü (Event Loop) Çalışma Mantığı

Mekanizma

- Dart varsayılan olarak tek iş parçacıklıdır (single-threaded).
- Asla **bloke** olmaz; olayları geldikleri sıraya göre (FIFO - İlk Giren İlk Çıkar) tek tek işler.
- Tüm Dart kodu varsayılan **'Ana Isolate'** içinde yaşar ve başlar.

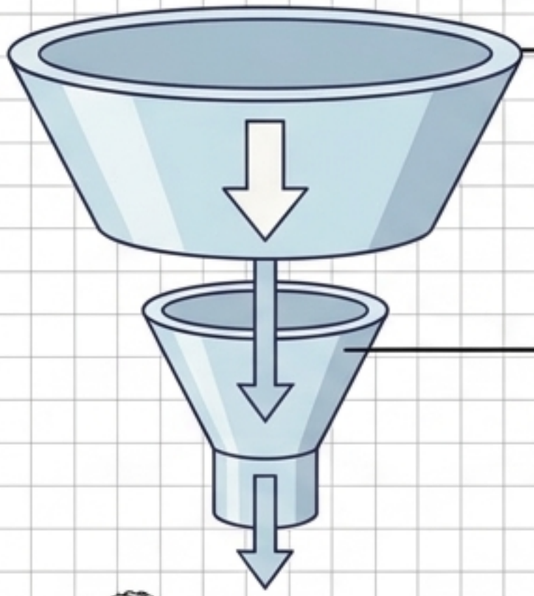


```
while (eventQueue.waitForEvent()) {
  eventQueue.processNextEvent();
}
```

Gemini Notebook

Kuyruk Öncelikleri: Event Queue vs Microtask Queue

Öncelik Hiyerarşisi



1. Microtask Queue

En yüksek öncelik. Hemen şimdi, mevcut işlem bitmeden önce yapılması gerekenler.

2. Event Queue

UI tıklamaları, I/O işlemleri, Future tamamlanmaları.

Dikkat! Microtask kuyruğunu tıkarsan, Event kuyruğundaki hiçbir şey (kullanıcı dokunuşları dahil) işlenemez.

```
void main() {
  Future(() => print('Event Queue 1'));
  Future.microtask(() => print('Microtask 1'));

  print('Senkron İşlem');
}
```

Konsol Çıktısı (Adım Adım):

1. **Senkron İşlem** (Ana kod bloğu hemen çalışır).
2. **Microtask 1** (Microtask kuyruğu, Event kuyruğundan önce boşaltılır).
3. **Event Queue 1** (Tüm microtask'ler bittikten sonra Event döngüsü başlar).

Gemini Notebook

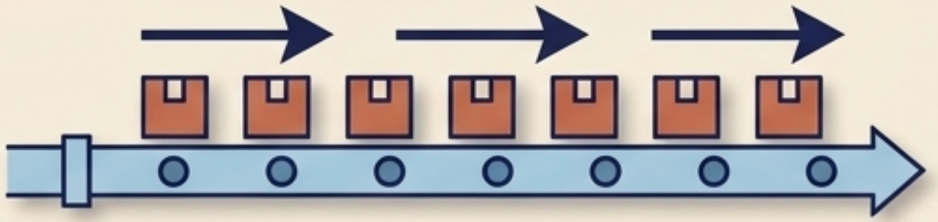
Asenkron Programlama Matrix'i: Synchronous vs Asynchronous

FUTURE

Tek Seferlik İşlem

`async` `await` `then()`

- **Kavram:** Gelecekte bir değer (veya hata) sağlama sözü.
- **Durumlar:** Uncompleted (Tamamlanmadı), Completed with Value (Değer), Completed with Error (Hata).
- **Kullanım:** HTTP ağ isteği, Dosya okuma.



STREAM

Sürekli Veri Akışı

`async*` `await for` `yield`

- **Kavram:** Zaman içinde art arda değerler sağlayan boru hattı. (Örn: Stream<int>)
- **Türler:** Single-subscription (Tek abonelikli) vs Broadcast (Çoklu yayın).
- **Kullanım:** Canlı veri (WebSockets), Kullanıcı arayüzü olayları.

Gemini Notebook

Future ve async/await Kullanımı: Hata Yönetimi

1.

- **async** ve **await**: Senkron koda benzeyen, okunabilir asenkron kod yazmayı sağlar.

2.

- **Future<String>**: Dönüş tipi, fonksiyonun er ya da geç bir String üreteceğini garanti eder.

```dart
const String filename = 'with_keys.json';

void main() async {
  try {
    final fileData = await _readFileAsync();
    final jsonData = jsonDecode(fileData);
    print('JSON anahtar sayısı: ${jsonData.length}');
  } catch (e) {
    print('Hata oluştu: $e');
  }
}

Future<String> _readFileAsync() async {
  final file = File(filename);
  final contents = await file.readAsString();
  return contents.trim();
}
```

3.

- **try-catch Blokları**: Asenkron execution sırasında fırlatılan çalışma zamanı (runtime) hatalarını zarifçe yakalar.

Gemini Notebook

Veri Akışları (Streams): Zaman İçinde Döngü Kurmak

1. **Stream Üretmek** (Generator Yöntemi):

```
Stream<int> streamGenerator(int max) async* {
  for (int i = 1; i <= max; i++) {
    await Future.delayed(Duration(seconds: 1));
    yield i; // Her saniye yeni bir değer yayar
  }
}
```

2. **Stream Tüketmek** (Asenkron Döngü):

```
Stream<int> sumStream(Stream<int> stream) async* {
  var sum = 0;
  await for (final value in stream) {
    sum += value;
    yield sum; // Yeni toplamı stream'e aktar
  }
}
```

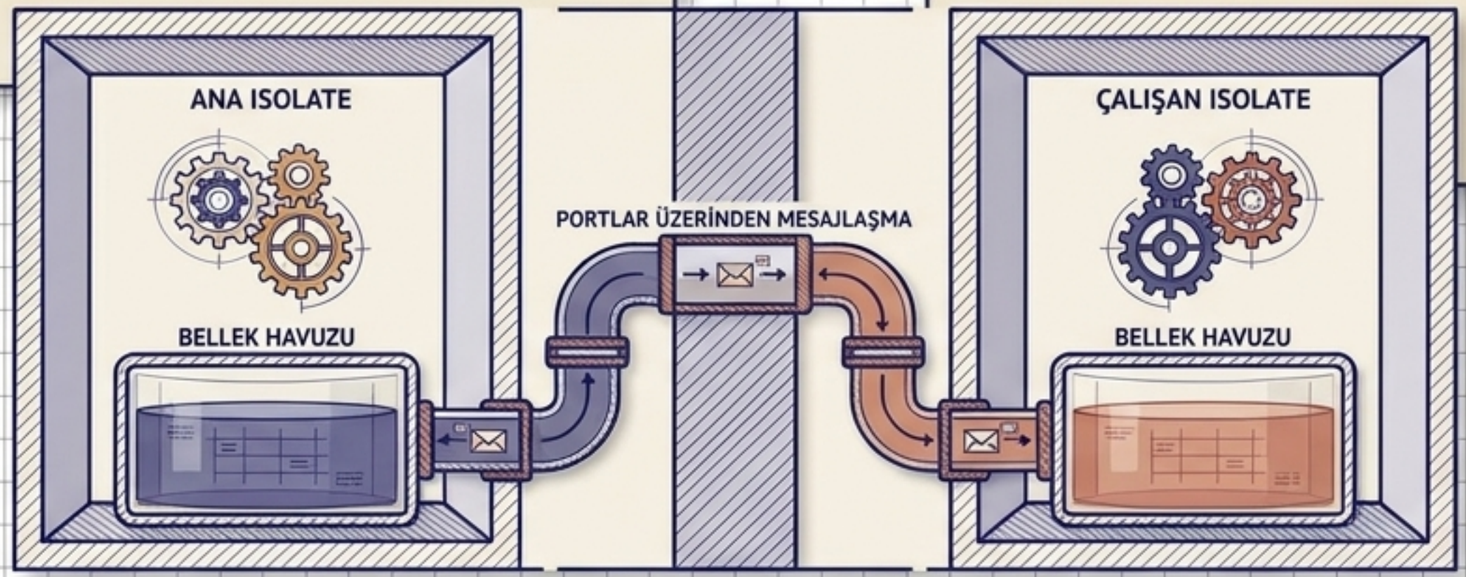
await for, UI dinleyicileri gibi sonsuz olay akışları için genellikle KULLANILMAMALIDIR. Gerçekten tüm sonuçların bitmesini beklediğiniz durumlarda tercih edin.

Gemini Notebook

Isolate Modeli ve Paylaşımsız Bellek (Shared Memory Yokluğu)

- **Kilitlenmeyi Aşmak:** Geleneksel iş parçacıklarının (threads) aksine, Isolate'lar bellek paylaşmaz.

- **Sıfır Veri Yarışı:** Paylaşılan durum (shared state) olmadığı için Mutex, Lock veya Data Race (Veri Yarışı) gibi eşzamanlılık karmaşıklıkları Dart'ta yaşanmaz.



- **Mutlak Yalıtım:** Her isolate'ın kendine ait belleği ve kendi Olay Döngüsünü çalıştıran tek bir iş parçacığı vardır.

Isolate'lar birbirlerinin hafızasının içine 'bakamazlar'. Bu, 'izolat' adının kökenini oluşturur.

Gemini Notebook

Hangi Isolate Stratejisi Tercih Edilmeli?

İşlemin süresi ve iletişim ihtiyacı nedir?

Tek Seferlik, Ağır Bir İşlem

`Isolate.run()`

- **Karakteristik:** Hızlıca arka planda çalışır, sonucu ana isolate'a döndürür ve belleği kopyalamadan verimli şekilde aktararak kendi kendini imha eder.
- **Kullanım:** Büyük JSON parse etme, dosya sıkıştırma.

Sürekli İletişim, Uzun Ömürlü Görev

`Isolate.spawn()`

- **Karakteristik:** Çift yönlü iletişim kanalları (Portlar) kurar. Uzun süre hayatta kalır ve arka planda çoklu mesaj işler.
- **Kullanım:** Sürekli ağ istekleri dinleme, karmaşık veritabanı senkronizasyonu.

Gemini Notebook

Kısa Ömürlü Çalışan: Isolate.run() Kod Örneği

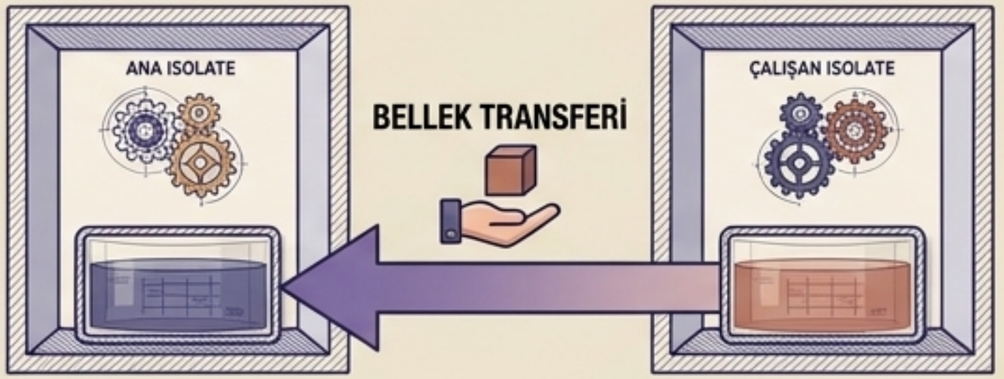
```dart
const String filename = 'massive_data.json';

void main() async {
  // Arka planda çalışan bir izole başlatılır
  final jsonData = await Isolate.run(_readAndParseJson);
  print('JSON anahtar sayısı: ${jsonData.length}');
}

// Tamamen yalıtılmış olarak çalışacak fonksiyon
Future<Map<String, dynamic>> _readAndParseJson() async {
  final fileData = await File(filename).readAsString();
  final parsed = jsonDecode(fileData) as Map<String, dynamic>;
  return parsed;
}
```

Mimari Avantaj:

Çalışan isolate, JSON ayrıştırma sonucunu tutan belleği ana isolate'a **AKTARIR**. Verileri pahalı pahalı bir şekilde kopyalamaz, sahipliğini devreder.



Gemini Notebook

Uzun Ömürlü Isolate İletişimi: Port Tabanlı Haberleşme

Ana Isolate

ReceivePort
(Posta Kutusu)

SendPort
(Postacı)

Worker Isolate

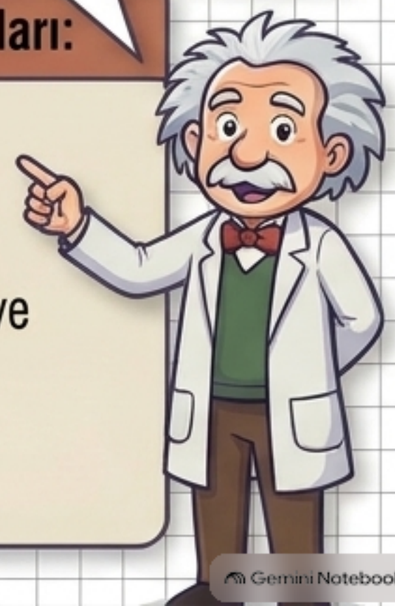
Isolate'lar birbirleriyle ancak 'SendPort' üzerinden mesaj geçirerek konuşabilir. Bu süreç FFI haricinde asenkrondur.

İletişim Kanallarının Anatomisi:

- **ReceivePort** (Posta Kutusu): Diğer isolate'lardan gelen mesajları dinleyen (stream benzeri) nesne.
- **SendPort** (Postacı): Sadece bağlı olduğu ReceivePort'a mesaj gönderebilen referans nesnesi.

Çift Yönlü İletişim Kurulum (Handshake) Aşamaları:

1. Ana isolate bir **ReceivePort** oluşturur.
2. Bu portun **SendPort**'u, **Isolate.spawn** ile yeni **isolate**'a ilk argüman olarak gönderilir.
3. Yeni isolate kendi **ReceivePort**'unu yaratır ve **SendPort**'unu ana **isolate**'a geri yollar.
4. Artık her iki taraf da birbirine mesaj iletebilecek açık kanallara sahiptir.



Gemini Notebook

Worker Mimarisi: Çoklu Mesaj İşleme ve Completer

```dart
class Worker {
  final SendPort _commands;
  final Map<int, Completer<Object?>> _activeRequests = {}; // Aktif İstek Haritası (Map)
  int _idCounter = 0;
  bool _closed = false;

  Future<Object?> parseJson(String message) async {
    if (_closed) throw StateError('Port Kapalı');

    final completer = Completer<Object?>.sync(); // İstek Tamamlayıcı (Completer)
    final id = _idCounter++;
    _activeRequests[id] = completer;
    _commands.send((id, message)); // ID ve mesajı (record olarak) gönder

    return await completer.future; // İşçiden yanıt gelene kadar bekle
  }
}
```

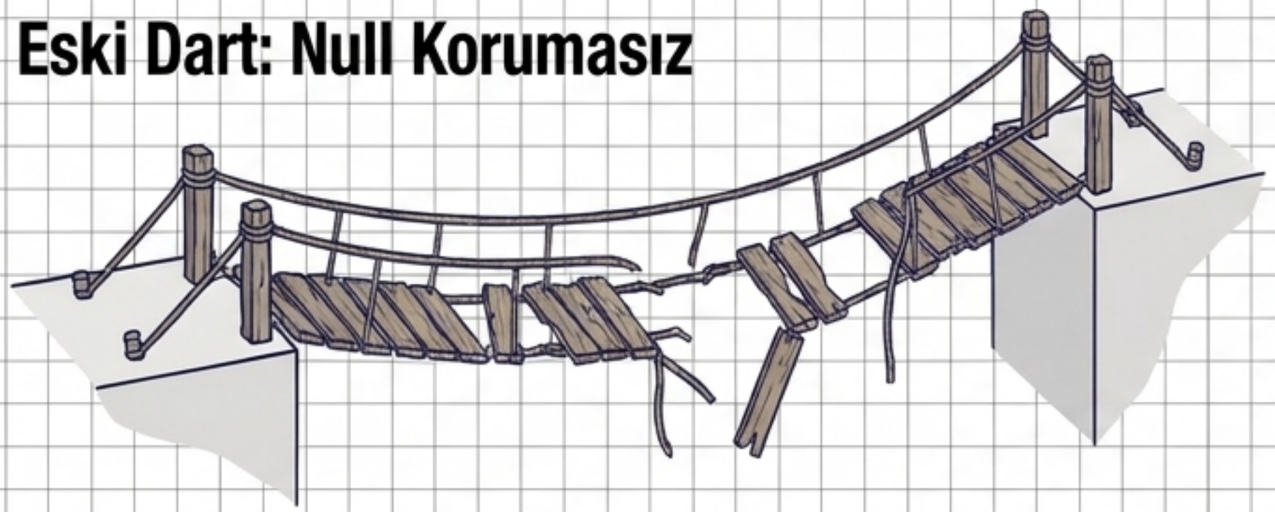
Mimari Odak:

Asenkron mesajların karışmamasını sağlamak için her isteğe benzersiz bir **ID** atanır ve **Completer** nesneleri ile yönetilir.

Gemini Notebook

Milyar Dolarlık Hata ve Sağlam Null Güvenliği

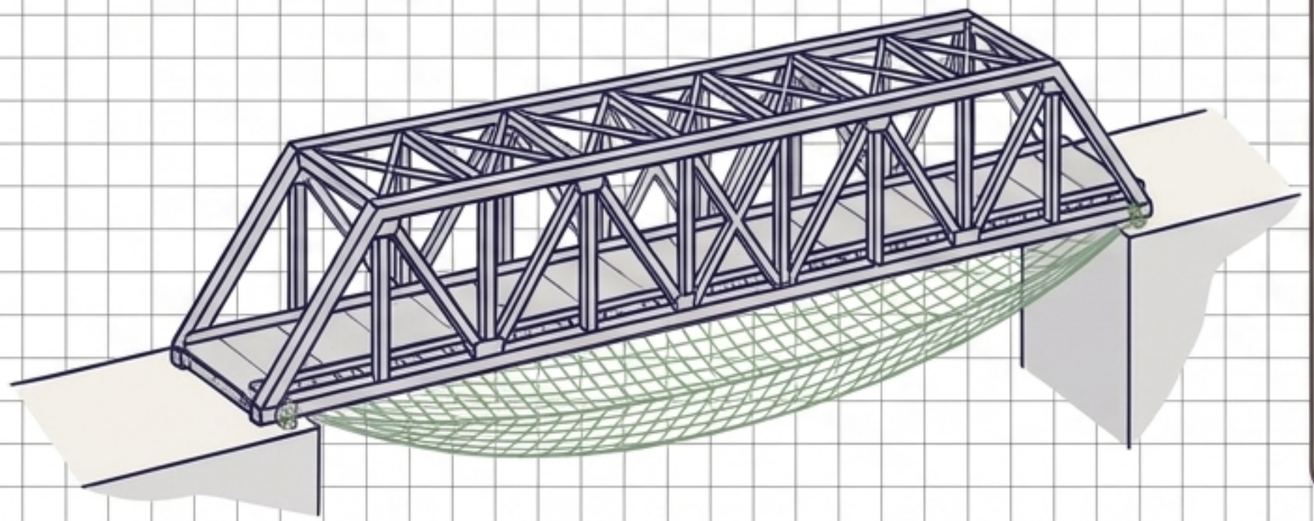
Eski Dart: Null Korumasız



Eski Dart (Null Korumasız):

- Tüm tipler varsayılan olarak Null değerini kabul edebilirdi.
- Çalışma zamanında (runtime) sık görülen **NoSuchMethodError** kilitlenmeleri.

Modern Dart: Sağlam Null Güvenliği

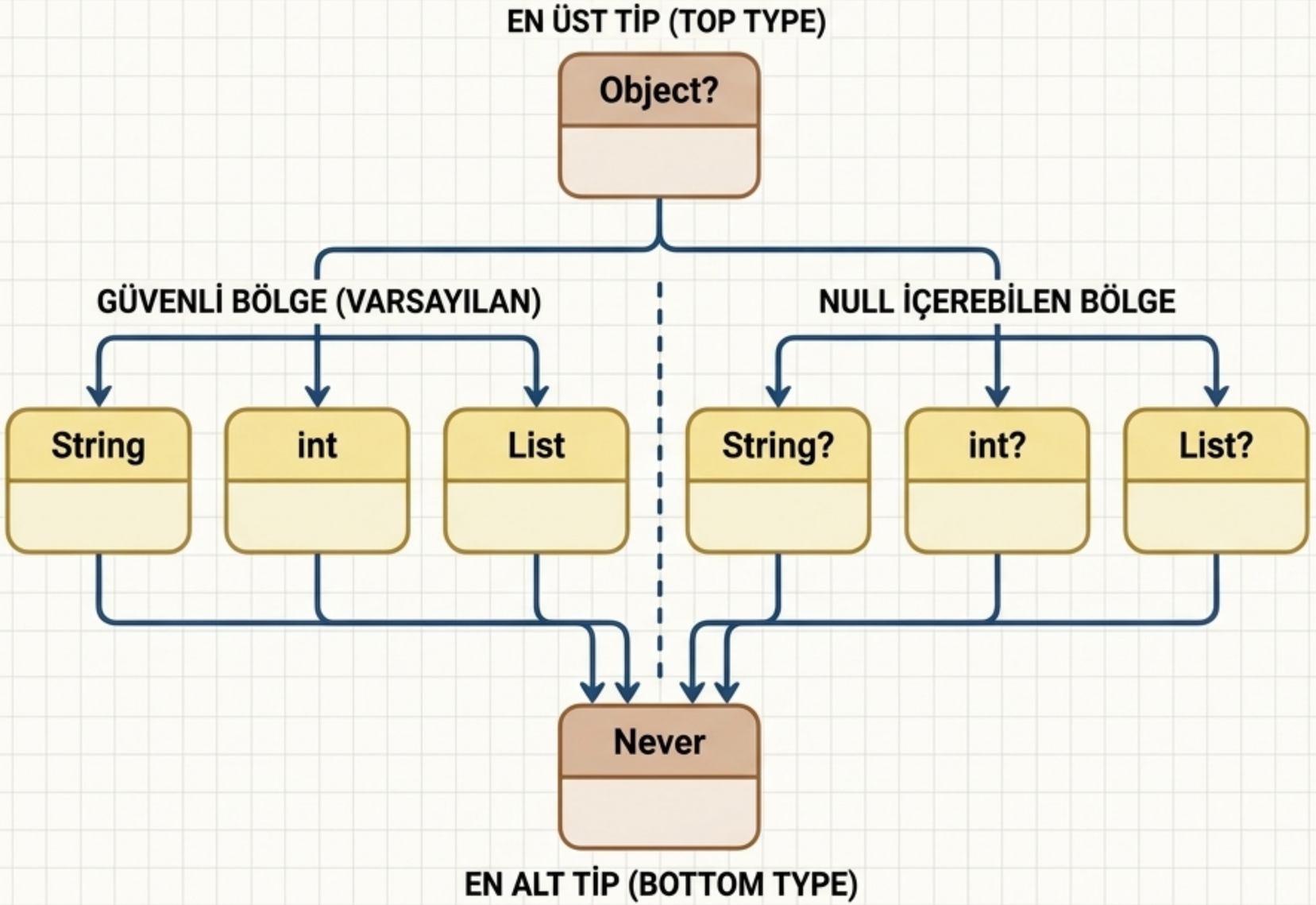


Modern Dart (Sound Null Safety):

1. **Varsayılan Olarak Null Olamaz:** Tür bildirimine '?' eklemediğiniz sürece hiçbir değişken null olamaz.
2. **Tamamen Sağlamlık** (Soundness): Tip sistemi bir değişkenin null olamayacağını belirlerse, derleyici bunun çalışma zamanında ASLA null olmayacağını garanti eder.

Gemini Notebook

Dart Tip Evreninin Bölünmesi (Type Hierarchy)



Zirve ve Dip Noktaları:

- **Top Type (Object?):** Hiyerarşinin en tepesi. Eskiden **Object** idi, artık tüm değerleri (null dahil) kapsayan top type **Object?** olmuştur.
- **Güvenli Bölge (Non-Nullable): String**, **int**, **List**. Her zaman metotlara erişilebilir, derleyici güveni tamdır.
- **Null İçerebilen Bölge (Nullable): String?**, **int?**. Doğrudan metot çağrılamaz, kontrol (check) mekanizması zorunludur.
- **Bottom Type (Never):** En alt tip. Hiçbir değeri temsil etmez, asla başarılı şekilde geri dönmeyecek fonksiyonların tipidir.

Gemini Notebook

Derleyici Akış Analizi ve Otomatik Tip Yükseltme

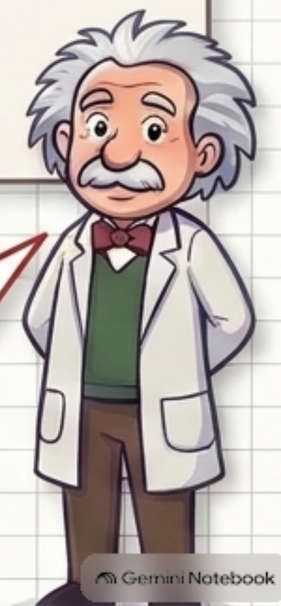
Akış Analizi (Flow Analysis) Zekası:

- **Reachability**: Fonksiyon sonundaki return ifadelerine ulaşılamayan yolları (örn: throw yapan bloklar) anlar.
- **Definite Assignment:** Null olamayan bir değişkenin kullanılmadan önce kesinlikle atanmış kesinlikle atanmış (initialized) olmasını denetler.

Type Promotion (Otomatik Tip Yükseltme):

```dart
String makeCommand(String executable, [List<String>? arguments]) {
  var result = executable;
  // arguments burada List<String>? (Nullable) tipindedir
  if (arguments != null) {
    // Derleyici null olmadığını kanıtlar ve otomatik promote eder
    result += ' ' + arguments.join(' '); // arguments artık List<String>
  }
  return result;
}
```

Önemli: Class alanları (fields) otomatik promote olmaz! Sadece yerel değişkenler (**local variables**) veya Dart 3.2+ ile **private final alanlar** bu avantaja sahiptir.



Gemini Notebook

Mimari Araçlar: 'late' ve 'required' Anahtar Kelimeleri

1. 'late' ile Tembel Başlatma (Lazy Initialization)

- **Kavram:** Derleyiciye ‘Bu değişkene güven, kullanılmadan önce kesinlikle değer atayacağım’ deme şeklidir.
- **Risk:** Atama yapılmadan okunursa LateInitializationError fırlatır.
- **Tembel Yürütme: late final int temp = _readThermometer();** Sadece değişkene ilk erişildiğinde fonksiyonu çalıştırır.

2. 'required' ile Zorunlu İsimlendirilmiş Parametreler

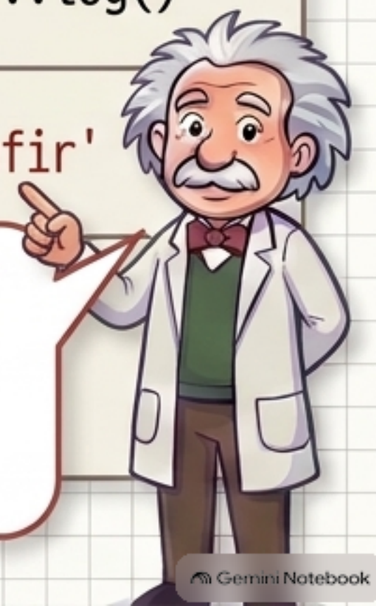
- **Kavram:** Named parameter’lar (süslü parantez içi {}) varsayılan olarak isteğe bağlıdır. Null olamayan türlerde, kullanıcıyı parametre girmeye zorlamak için kullanılır.

```
void setupServer({required String host, required int port, String? token}) {
  // host ve port zorunludur, token isteğe bağlıdır (nullable).
}
```

Gemini Notebook

Null-Aware Operatörler: İsviçre Çakısı Araçları

Operatör	İsim	Kullanım Mantığı & Kısa Devre (Short-Circuit)	Örnek Kod
?.	Null-Aware Access	Nesne null ise işlemi atlar, hata fırlatmak yerine null döndürür.	`thing?.doohickey.gizmo`
?[]	Null-Aware Indexing	Liste/Map null ise index aramaya çalışmaz, null döndürür.	`jsonArray?[0]`
?..	Null-Aware Cascade	Nesne null değilse art arda metot zincirlemesi yapar.	`user?..save()..log()`
??	Null-Coalescing	Sol taraf null ise sağ taraftaki yedek (fallback) değeri atar.	`name ?? 'Misafir'`
!	Null Assertion (Bang)	Derleyiciyi ezer! 'Bu kesin null değil' der. Yanılırsanız çöker.	



Metod zincirlerinde ?. operatörü kısa devre (short-circuit) yapar. İlk **?. null** dönerse, sağındaki tüm işlemler anında iptal edilir.

Gemini Notebook

Dil Sürümleme (Language Versioning) Neden Gereklidir?

Kavramsal Temel:

- **Geriye dönük uyumsuz büyük değişikliklerde** (örn. Null Safety), Dart mevcut kodların kırılmasını önlemek için dosyalar arası veya paketler arası sürüm yönetimine izin verir.
- **Karma sürümlü** (mixed-version) projeleri destekler: Yeni null-safe kod ile eski (legacy) kod aynı projede çalışabilir.

Sürüm Bildirim Yöntemleri:

1. Dosya Bazında Bildirim (Header):

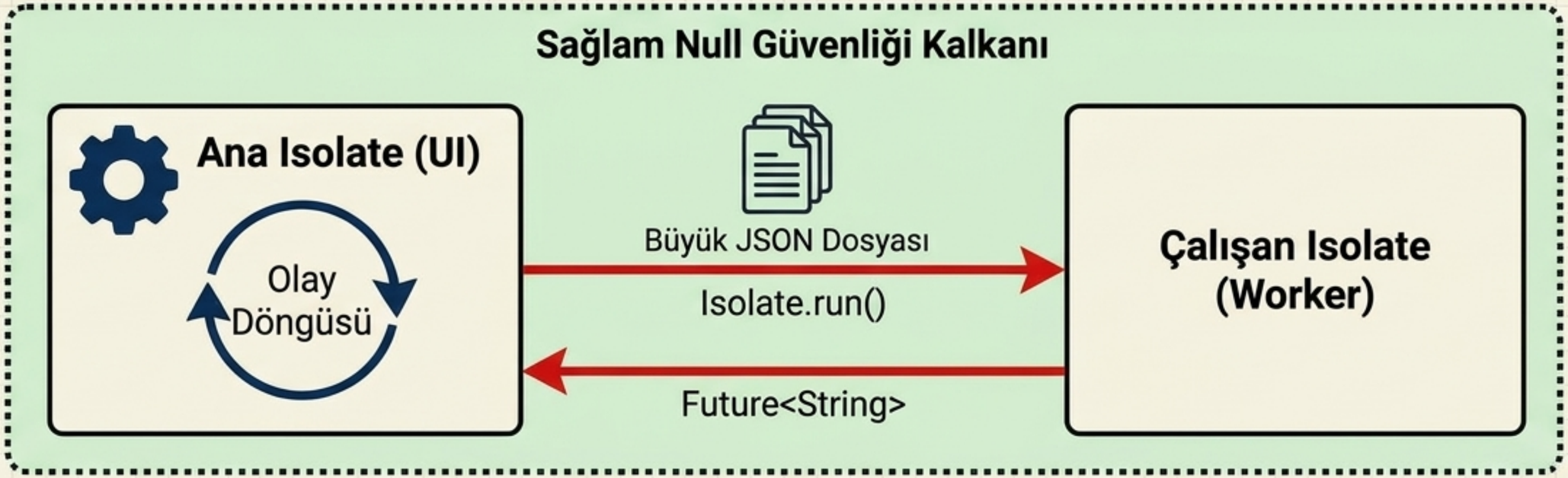
```
// @dart=2.17
class LegacyCode { ... }
```

2. Proje Bazında (pubspec.yaml SDK Constraints):

```
environment:
  sdk: ">=3.0.0 <4.0.0" # Dart 3 ile
tam sağlam null güvenliği zorunludur
```

Gemini Notebook

SENTEZ: Modern Bir Dart Mimarisi



Büyük Resim (Mükemmel Uyum):

- **UI ve Event Loop:** Ana Isolate (UI), Event Loop sayesinde kullanıcı etkileşimlerini kesintisiz ve akıcı bir şekilde, bloklanmadan işler.
- **Isolate Worker Ağları:** Ağır JSON parse etme veya veritabanı işlemleri, Isolate.run() veya uzun ömürlü Port iletişimleri ile arka plana devredilir.
- **Sağlam Null Güvenliği Kalkanı:** Tüm veri akışları, statik tip sisteminin katı derleyici kuralları (Non-Nullable varsayılanı, Type Promotion) ile çökmelere karşı sigortalanır.

Gemini Notebook

İleri Düzey Dart: Karşılaştırmalı Özet Matrisi

Asynchronous (Tek İplik)

- **Future** (Tek değer), **Stream** (Sürekli akış), **async/await, Event Loop**. Hafiftir, I/O için idealdir.

Isolates (Çok Çekirdek)

- Paylaşımsız bellek, **SendPort/ReceivePort** mesajlaşması. CPU yoğun, bloklayıcı algoritmalar için zorunludur.

Nullable Tents (?)

- **String?**, Başlangıç değeri gerektirmez, doğrudan metot çağrılamaz, ?. veya kontrol gerektirir.

Non-Nullable & Soundness

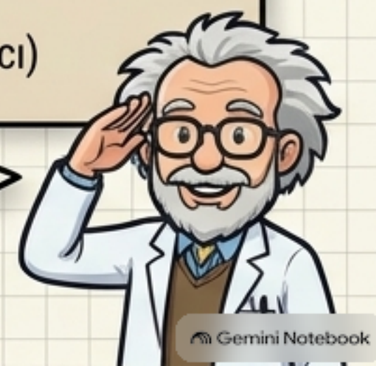
- **String, late, required, Type Promotion**. Derleme zamanı güvenliği sağlar, NoSuchMethodError hatasını yok eder.

Non-Nullable & Yenlıpt

- **String, late, required!, Type Promotion**. Derleme zamaı güvenliği sağlar, late olsur NoSuchMethodEror hatasını yok eder.

SCÜ Mobil Programlama I Dersi Kapsamında Hazırlanmıştır.

Teşekkürler | **Takım: Toys Code**
(Ali Akbaş, Emrullah Altay, Emirhan Yazıcı)



Gemini Notebook